Rockchip GPIO Development Guide

ID: RK-YH-YF-803

Release Version: V1.0.1

Release Date: 2025-11-28

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document mainly introduces the usage of GPIO in Kernel and U-Boot, GPIO debugging tools, and common troubleshooting methods for GPIO issues.

Product Version

Chip Models
All Rockchip Platforms

Target Audience

This document (this guide) is mainly intended for the following engineers:

Technical Support Engineers

Software Development Engineers

Revision History

Version Number	Author	Modification Date	Modification Description
V1.0.0	Shen Tao	2024-08-08	Initial version
V1.0.1	Xu Minghao	2025-11-28	Added methods for IO multiplexing, reading level anomalies, and improving interrupt efficiency

Table of Contents

Rockchip GPIO Development Guide

1. IO-Domain
    1.1 Kernel Configuration
    1.2 U-Boot Configuration
2. Developing GPIO in Kernel
    2.1 GPIO Multiplexing
    2.2 GPIO Input/Output
    2.3 Dynamic IO Multiplexing Configuration
3. GPIO Development in U-Boot
    3.1 U-Boot LED Driver Development
    3.2 U-Boot GPIO Driver Development
4. User-space GPIO Development
5. IO Sleep Wakeup Configuration
    5.1 Configuring GPIO Wakeup
    5.2 Configuring GPIO Levels During Sleep
6. GPIO Expansion Chips
7. GPIO Simulating I2C
8. GPIO Debug Tools
    8.1 IO Tools
        8.1.1 Kernel Phase IO Tools
        8.1.2 U-Boot Phase IO Tools
    8.2 IOMUX Tool
    8.3 Pinconfig Tool
    8.4 PinDebug Tool
9. GPIO Troubleshooting Steps
    9.1 Power Confirmation
    9.2 IO Multiplexing Verification
    9.3 GPIO CLK Status Confirmation
    9.4 Conflict Verification Caused by Other Drivers Calling GPIO
    9.5 GPIO Register Verification
10. Frequently Asked Questions
    10.1 Inability to Control System Power-On Output Voltage Levels
    10.2 Erroneous GPIO Interrupt Triggering
    10.3 Inability to Control RV1103/RV1106's gpio3b0~gpio3c3
    10.4 Inability to Control RV1108's gpio3b0 to gpio3c3
    10.5 RK3588's gpio4d2 and gpio4d3 Inoperable
    10.6 Android SDK Unable to Locate System-Level Interface for /sys/class/gpio
    10.7 Abnormal GPIO Level Reading
    10.8 GPIO Interrupt Binding to a Specific Core
    10.9 GPIO Interrupt Response Latency

1. IO-Domain

The general output voltage levels of IO are 1.8v, 3.0v, 3.3v, etc., and different output voltages depend on the power domain voltage where the IO is located, which is related to the hardware power supply. The io-domain is a software configuration item for the IO power domain register. If the power domain software configuration does not match the actual hardware, it will cause the IO port in this power domain to be uncontrollable, resulting in disordered signals and non-compliant outputs. If, during actual development, an abnormality is encountered in a certain IO, and the pin-mux status is correct without being called elsewhere, but the measured signal is incorrect, and the software cannot control the output level, it is necessary to check the configuration of the power domain register for that IO. For detailed development guidelines on io-domain, please refer to the documentation included in the SDK.

Android SDK file path: RKDocs/common/IO-Domain

Linux SDK documentation path: docs/cn/Common/IO-DOMAIN

Note: Some SDKs do not include development documentation for io-domain. Please refer to the following instructions or contact the corresponding FAE window for assistance.

It is particularly important to note that new chip models from Rockchip (such as: RK3588, RK3562, RV1106, RV1103, etc.) do not require software configuration for io_domains. The chip can internally adapt to voltage domains, and this can be confirmed by the default compiled EVB board firmware in the kernel, where hidden files with the .xxx.dtb.dts.tmp extension are saved. If no io-domain configuration items are found in the tmp file, it indicates that software configuration for the voltage domain is not necessary.

1.1 Kernel Configuration

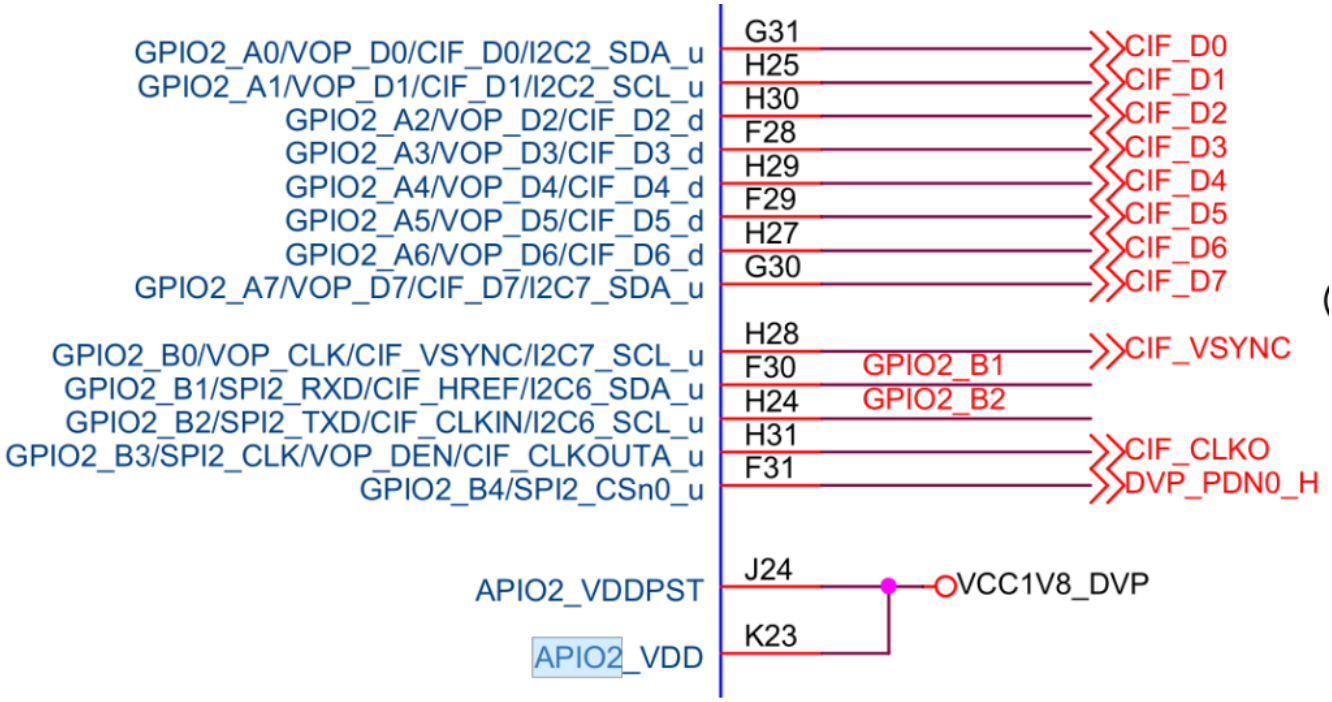
Here, taking RK3399 as an example, open the GRF/PMUGRF section of the TRM specification manual corresponding to the RK3399 main control chip and search for keywords such as "VSEL", "volsel", and "vsel" to locate the io-domain registers.

GRF_IO_VSEL

Address: Operational Base + offset (0x0e640)

Bit	Attr	Reset Value	Description
31:16	RW	0x0000	write_enable bit0~15 write enable When bit 16=1, bit 0 can be written by software . When bit 16=0, bit 0 cannot be written by software; When bit 17=1, bit 1 can be written by software . When bit 17=0, bit 1 cannot be written by software; ..... When bit 31=1, bit 15 can be written by software . When bit 31=0, bit 15 cannot be written by software;
15:4	RO	0x0	reserved
3	RW	0x0	gpio1830_gpio4cd_ms 1'b0: IO domain works as 3.0V; 1'b1: IO domain works as 1.8V;
2	RW	0x0	sdmmc_gpio4b_ms 1'b0: IO domain works as 3.0V; 1'b1: IO domain works as 1.8V;
1	RW	0x0	audio_gpio3d4a_ms 1'b0: IO domain works as3.0V; 1'b1: IO domain works as 1.8V;
0	RW	0x0	bt656_gpio2ab_ms 1'b0: IO domain works as 3.0V; 1'b1: IO domain works as 1.8V;
9	RW	0x1	pmu1830_vol pmu IO 1.8v/3.0v select. 0: IO domain works as3.0v ; 1: IO domain works as 1.8v ;

Identify the corresponding registers through the keywords. Then, examine the power supply corresponding to the pins on the schematic diagram, for example, the control of the entire row of pins for gpio2 is abnormal:



The power supply corresponding to this row of pins is APIO2. Next, go to the kernel directory in the sdk code to find the document: kernel/Documentation/devicetree/bindings/power/rockchip-io-domain.txt. This document provides a detailed description of the definition of APIO2 power supply in the software:

```
Possible supplies for rk3399:
- bt656-supply:  The supply connected to APIO2_VDD.
- audio-supply:  The supply connected to APIO5_VDD.
- sdmmc-supply:  The supply connected to SDMMC0_VDD.
- gpio1830-supply:  The supply connected to APIO4_VDD.
Possible supplies for rk3399 pmu-domains:
- pmu1830-supply:The supply connected to PMUIO2_VDD.
```

Search for "io_domain" in the dts directory to find:

```
&io_domains {
    status = "okay";

    bt656-supply = <&vcc1v8_dvp>;
    audio-supply = <&vcca1v8_codec>;
    sdmmc-supply = <&vcc_sdio>;
    gpio1830-supply = <&vcc_3v0>;
};
```

Ensure that the power supply settings in the software configuration match the hardware:

```
bt656-supply = <&vcc1v8_dvp>;
```

Check the debug node information:

```
console:/ # cat /sys/kernel/debug/regulator/regulator_summary
```

The setting of 1.8V here should be consistent with the hardware schematic diagram to be correct; if not, change it to be consistent.

1.2 U-Boot Configuration

Refer to section 9.13 of the document "RKDocs/rk3399/Rockchip_RK3399_Developer_Guide_Android7.1_Software_CN&EN.pdf". Add rkdevelop branch U-Boot configuration io-domain functionality, which addresses the following issues:

- Solves issues caused by incorrect IO voltage levels or iomux settings due to io-domain errors in U-Boot.
- If the io-domain is not set during the U-Boot phase, the chip will load the default configuration until the kernel loads the correct DTS configuration, which may take approximately 3-4 seconds. If an io-domain is connected to 3.0V but the default configuration is 1.8V, there will be a voltage mismatch that could potentially damage the RK3399 IO.

Ensure that the Kernel's io-domain is configured correctly, and confirm that U-Boot and Kernel have the following two commits: (u-boot/drivers/power/rockchip-io-domain.c)

```
u-boot:
commit 5a4e354a116b45b8ac2311aaa3464a7e88cd13d5
Author: Jianqun Xu <jay.xu@rock-chips.com>
Date: Thu Mar 19 15:38:19 2020 +0800
power: support rockchip io-domain set
This patch should work with kernel dtb node property - "uboot-set" in io-domain node or
pmu-io-domain node.
Change-Id: I86fa43d234091310ee08978ec42f5895fb010a8c
Signed-off-by: Jianqun Xu <jay.xu@rock-chips.com>
Signed-off-by: Wenping Zhang <wenping.zhang@rock-chips.com>
kernel:
commit f95ad65aabffa7450ff90a0dd4778cd5500de937
Author: Jianqun Xu <jay.xu@rock-chips.com>
Date: Fri Mar 20 09:52:57 2020 +0800
arm64: dts: rockchip: add 'uboot-set' in io-domain node for rk3399.
Add 'uboot-set' for io-domain node, which used by u-boot board init to set io domain.
Change-Id: Ia897e240c04d0b362551952278dadc8b4f2a8f2f
Signed-off-by: Jianqun Xu <jay.xu@rock-chips.com>
Signed-off-by: Wenping Zhang <wenping.zhang@rock-chips.com>
```

2. Developing GPIO in Kernel

This section primarily discusses the configuration and methods for developing GPIO in the Kernel.

2.1 GPIO Multiplexing

DTS Configuration

Taking the RK3588 Android12 platform as an example, the configuration is as follows. To operate the gpio3b7 pin, configure `enable-gpios = <&gpio3 RK_PB7 GPIO_ACTIVE_HIGH>` in the DTS. To prevent the GPIO pin from being multiplexed to other functions before the Kernel starts, which could prevent GPIO control, you can set the `pinctrl-names` and `pinctrl-0` properties to multiplex the pin to a GPIO function. If a user needs to request an IO interrupt, gpio3b6 can be configured as a Schmitt input mode.

```
/{
    gpio_demo: gpio-demo {
        status = "okay";
        compatible = "rockchip,gpio-demo";
        enable-gpios = <&gpio3 RK_PB7 GPIO_ACTIVE_HIGH>;
        irq-gpios = <&gpio3 RK_PB6 GPIO_ACTIVE_HIGH>;
        pinctrl-names = "default";
        pinctrl-0 = <&gpio_pin &gpio_irq>;
    };
};

&pinctrl {
    gpio-pin {
        gpio_pin: gpio-pin {
            // Configure gpio3b7 iomux to GPIO
            rockchip,pins = <3 RK_PB7 RK_FUNC_GPIO &pcfg_pull_none>;
        };
    };

    gpio-irq {
        gpio_irq: gpio-irq {
            // Configure gpio3b6 iomux to GPIO and Schmitt input mode
            rockchip,pins = <3 RK_PB6 RK_FUNC_GPIO &pcfg_pull_none_smt>;
        };
    };
};
```

The `pinctrl-0` referenced property node follows these rules:

- Must be under the pinctrl node;
- Must be added in the form of function+group;
- Follow other basic rules of DTS;
- The format of function+group is as follows:

```
function {
    group {
        rockchip,pins = <gpiobank gpionum iomuxfunc &config>;
    };
};
```

`iomuxfunc` refers to the multiplexing function, `RK_FUNC_GPIO` is configured to multiplex to GPIO function. `config` is for configuring IO pin pull-up, pull-down, port Schmitt trigger mode, and drive strength. Refer to the SDK file rockchip-pinconf.dtsi, RK lists all configurations in the DTS pinctrl node. As follows:

```
&pinctrl {
```

```
    pcfg_pull_up: pcfg-pull-up {
        bias-pull-up;
    };

    pcfg_pull_down: pcfg-pull-down {
        bias-pull-down;
    };

    pcfg_pull_none: pcfg-pull-none {
        bias-disable;
    };

    pcfg_pull_none_smt: pcfg-pull-none-smt {
        bias-disable;
        input-schmitt-enable;
    };

    pcfg_pull_none_drv_level_0: pcfg-pull-none-drv-level-0 {
        bias-disable;
        drive-strength = <0>;
    };

    ......
    pcfg_output_high: pcfg-output-high {
        output-high;
    };

    ......
    pcfg_output_low: pcfg-output-low {
        output-low;
    };

    ......
};
```

Where `drive-strength = <2>` indicates the drive strength level is 2, and the value written to the drive strength register is 2. Taking RK3588 as an example, the specific drive strength level of gpio2a0 is shown in the following figure, where level 2 represents 50ohm.

RK3588 TRM

Bit	Attr	Reset Value	Description
2:0	RW	0x6	gpio2a0_ds GPIO2A0 DS control Driver Strength Selection 3'b000: 100ohm 3'b001: 66ohm 3'b010: 50ohm 3'b011: 40ohm 3'b100: 33ohm 3'b101: 25ohm

User Driver Code

Taking the RK3588 Android12 platform as an example, after the DTS configuration is completed, the user driver code obtains the DTS configuration to operate GPIO. The user driver code can apply for and operate GPIO through the following functions.

```
// Get the GPIO device. If multiple devices are used, the index parameter must be included
struct gpio_desc *gpiod_get(struct device *dev, const char *con_id,
                    enum gpiod_flags flags)

struct gpio_desc *gpiod_get_index(struct device *dev,
                      const char *con_id, unsigned int idx,
                      enum gpiod_flags flags)
// Release the GPIO device
void gpiod_put(struct gpio_desc *desc)
void gpiod_put_array(struct gpio_descs *descs)

// Set gpio input/output
int gpiod_direction_input(struct gpio_desc *desc)
int gpiod_direction_output(struct gpio_desc *desc, int value)

// Get/Set gpio value
int gpiod_get_value(const struct gpio_desc *desc);
void gpiod_set_value(struct gpio_desc *desc, int value);

// Get the GPIO IRQ interrupt number
int gpiod_to_irq(const struct gpio_desc *desc);
......
```

User driver operation GPIO code reference is as follows:

```
// SPDX-License-Identifier: GPL-2.0
/*
 * Copyright (c) 2020 Rockchip Electronics Co. Ltd.
```

```
 *
 * Author: Johnny Shi <nengqiang.shi@rock-chips.com>
 */

/* The following code is based on the RK3588 Android 12 platform */
#include <linux/gpio/consumer.h>
#include <linux/interrupt.h>
#include <linux/kernel.h>
#include <linux/math64.h>
#include <linux/module.h>
#include <linux/of_graph.h>
#include <linux/phy/phy.h>
#include <linux/platform_device.h>

static irqreturn_t irq_demo_handler(int irq, void *dev_id)
{
    printk("------irq_demo------\n");

    return IRQ_HANDLED;
}

static int gpio_demo_probe(struct platform_device *pdev)
{
    struct gpio_desc *enable_gpio, *irq_gpio;
    int irq;
    int ret;

    printk("------gpio_demo_probe------\n");

    // Get the gpio3b7 pin and set it as output
    enable_gpio = devm_gpiod_get_index(&pdev->dev, "enable", 0, GPIOD_OUT_HIGH);
    if (IS_ERR(enable_gpio)) {
        ret = PTR_ERR(enable_gpio);
        dev_err(&pdev->dev, "failed to request enable GPIO: %d\n", ret);
        return ret;
    }

    // Get the gpio3b6 pin and set it as input
    irq_gpio = devm_gpiod_get_index(&pdev->dev, "irq", 0, GPIOD_IN);
    if (IS_ERR(irq_gpio)) {
        ret = PTR_ERR(irq_gpio);
        dev_err(&pdev->dev, "failed to request irq GPIO: %d\n", ret);
        return ret;
    }
    // Get the gpio3b6 IRQ interrupt number and request IRQ
    irq = gpiod_to_irq(irq_gpio);
    ret = request_irq(irq, irq_demo_handler, IRQF_TRIGGER_FALLING, "demo_irq", NULL);
    if (ret) {
        dev_err(&pdev->dev, "failed to request IRQ: %d\n", ret);
        return ret;
    }

    // Set gpio3b7 pin output high
```

```
    gpiod_set_value(enable_gpio, 1);

    return 0;
}

static const struct of_device_id gpio_demo_of_match[] = {
    { .compatible = "rockchip,gpio-demo" },
    {}
};

static struct platform_driver gpio_demo_driver = {
    .driver = {
        .name = "gpio-demo",
        .of_match_table = of_match_ptr(gpio_demo_of_match),
    },
    .probe  = gpio_demo_probe,
//    .remove = gpio_demo_remove,
};
module_platform_driver(gpio_demo_driver);

MODULE_DESCRIPTION("Rockchip gpio demo driver");
MODULE_LICENSE("GPL v2");
```

Based on the DTS configuration, the user driver obtains the gpio3b7 pin through `devm_gpiod_get_index(&pdev->dev, "enable", 0, GPIOD_OUT_HIGH)`, and sets the pin to high-level effective according to the `GPIO_ACTIVE_HIGH` attribute in the DTS, while `GPIOD_OUT_HIGH` initializes gpio3b7 as an output. `gpiod_set_value(enable_gpio, 1)` causes gpio3b7 to output a high level. Referring to the above code, when a user applies for IRQ, they must first obtain the gpio3b6 pin through `devm_gpiod_get_index(&pdev->dev, "irq", 0, GPIOD_IN)` and initialize it as an input. By using `gpiod_to_irq(irq_gpio)`, `request_irq(irq, irq_demo_handler, IRQF_TRIGGER_FALLING, "demo_irq", NULL)` to apply for IRQ.

2.2 GPIO Input/Output

In the DTS, by configuring `pinctrl-names = "default"`, and then setting the GPIO properties to `pcfg_output_high` or `pcfg_output_low` in `pinctrl-0`, the GPIO can directly output high or low voltage levels. Since the driver will automatically configure the GPIO properties in `pinctrl-0` when it is loaded.

For example:

```
imx415: imx415@1a {
    compatible = "sony,imx415";
    reg = <0x1a>;
    clocks = <&cru CLK_MIPI_CAMARAOUT_M1>;
    clock-names = "xvclk";
    pinctrl-names = "default";
-   pinctrl-0 = <&mipim0_camera1_clk>;
+   pinctrl-0 = <&mipim0_camera1_clk &test_gpio>; //Add a test_gpio
    power-domains = <&power RK3588_PD_VI>;
    ......
```

```
}

&pinctrl {
    test {
        test_gpio: test-gpio {
            //GPIO3_B1 is configured as a high voltage output, it can also be configured as
pcfg_output_low for low voltage output.
            rockchip,pins = <3 RK_PB1 RK_FUNC_GPIO &pcfg_output_high>;
        };
    };
}
```

GPIO input configuration, generally set to `pcfg_pull_none` or `pcfg_pull_none_smt` will suffice.

2.3 Dynamic IO Multiplexing Configuration

Users can implement dynamic switching of IO pin multiplexing, such as using IO pins as UART during certain stages of kernel operation and as regular GPIO ports in other stages. Taking the RK3588 Android12 platform as an example, the configuration is as follows.

DTS Configuration

`pinctrl-names` In addition to the four states defined in the kernel, users can also define custom states. The `"default"` state sets the uart3 pins by default, and the kernel will automatically bind and set them after the user driver matches, eliminating the need for further settings by the user driver. For custom state pin settings, refer to `pinctrl-names= "iomux-exchange"`, which users can parse and dynamically configure pin multiplexing as needed in the user driver.

```
    iomux_exchange: iomux-exchange {
        compatible= "iomux-exchange-demo";
        pinctrl-names= "default", "iomux-exchange";
        pinctrl-0= <&uart3m0_xfer>;
        pinctrl-1= <&uart3m0_gpio>;
    };

    &pinctrl {
        uart3 {
            /omit-if-no-ref/
            uart3m0_xfer: uart3m0-xfer {
                rockchip,pins =
                    /* uart3_rx_m0 */
                    <1 RK_PC0 10 &pcfg_pull_up>,
                    /* uart3_tx_m0 */
                    <1 RK_PC1 10 &pcfg_pull_up>;
            };
            uart3-gpio {
                uart3m0_gpio: uart3m0-gpio {
                        rockchip,pins =
                                <1 RK_PC0 RK_FUNC_GPIO &pcfg_pull_none>,
                                <1 RK_PC1 RK_FUNC_GPIO &pcfg_pull_none>;
                };
```

```
            };
        };
    };
```

User Driver Code

In the user driver, `devm_pinctrl_get` is used to obtain the pinctrl resource, `pinctrl_lookup_state` to obtain the GPIO configuration of the user-defined state, and `pinctrl_select_state` to set the IO pin state. The user driver code reference is as follows:

```
// SPDX-License-Identifier: GPL-2.0
/*
 * Copyright (c) 2020 Rockchip Electronics Co. Ltd.
 *
 * Author:Johnny Shi <nengqiang.shi@rock-chips.com>
 */
/* The following code is based on the RK3588 Android 12 platform */
#include <linux/gpio/consumer.h>
#include <linux/interrupt.h>
#include <linux/kernel.h>
#include <linux/math64.h>
#include <linux/module.h>
#include <linux/of_graph.h>
#include <linux/phy/phy.h>
#include <linux/platform_device.h>
#include <linux/pinctrl/consumer.h>

struct pinctrl *uart_pinctrl;
struct pinctrl_state   *uart_default;//uart
struct pinctrl_state   *uart_gpio;//gpio

static int iomux_exchange(int select) {
    int ret = 0;
    if (select) {
        ret = pinctrl_select_state(uart_pinctrl, uart_gpio);
    } else {
        ret = pinctrl_select_state(uart_pinctrl, uart_default);
    }

    return ret;
}

static int iomux_exchange_demo_probe(struct platform_device *pdev)
{
    printk("------iomux_exchange_demo_probe------\n");
    uart_pinctrl = devm_pinctrl_get(&pdev->dev);
    if (IS_ERR_OR_NULL(uart_pinctrl)) {
        printk("Failed to get pin ctrl\n");
        return PTR_ERR(uart_pinctrl);
    }

    // Gets the pinctrl status property of "default"
    uart_default = pinctrl_lookup_state(uart_pinctrl, "default");
```

```
    if (IS_ERR_OR_NULL(uart_default)) {
        printk("Failed to lookup pinctrl default state\n");
        return PTR_ERR(uart_default);
    }

    // Get the custom "iomux-exchange" pinctrl status properties
    uart_gpio = pinctrl_lookup_state(uart_pinctrl, "iomux-exchange");
    if (IS_ERR_OR_NULL(uart_gpio)) {
        printk("Failed to lookup pinctrl iomux-exchange state\n");
        return PTR_ERR(uart_gpio);
    }

    iomux_exchange(1);
    //iomux_exchange(0);
    return 0;
}

static const struct of_device_id iomux_exchange_demo_of_match[] = {
    { .compatible = "iomux-exchange-demo" },
    {}
};

static struct platform_driver iomux_exchange_demo_driver = {
    .driver = {
        .name = "iomux-exchange-demo",
        .of_match_table = of_match_ptr(iomux_exchange_demo_of_match),
    },
    .probe  = iomux_exchange_demo_probe,
//    .remove = iomux_exchange_demo_remove,
};
module_platform_driver(iomux_exchange_demo_driver);

MODULE_DESCRIPTION("Rockchip iomux exchange demo driver");
MODULE_LICENSE("GPL v2");
```

If `iomux_exchange(1)` is not called in the above code, the pins will be configured as UART pins by default. After the system starts, the terminal command is used to view IO multiplexing:

```
console:/ # cat sys/kernel/debug/pinctrl/pinctrl-rockchip-pinctrl/pinmux-pins
......
pin 44 (gpio1-12): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 45 (gpio1-13): fde60000.dp gpio1:45 function dp group dp1-hpd
pin 46 (gpio1-14): 5-001a (GPIO UNCLAIMED) function mipi group mipim0-camera1-clk
pin 47 (gpio1-15): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 48 (gpio1-16): iomux-exchange (GPIO UNCLAIMED) function uart3 group uart3m0-xfer
pin 49 (gpio1-17): iomux-exchange (GPIO UNCLAIMED) function uart3 group uart3m0-xfer
pin 50 (gpio1-18): 7-0011 (GPIO UNCLAIMED) function i2s0 group i2s0-mclk
pin 51 (gpio1-19): fe470000.i2s (GPIO UNCLAIMED) function i2s0 group i2s0-idle
pin 52 (gpio1-20): (MUX UNCLAIMED) gpio1:52
pin 53 (gpio1-21): fe470000.i2s (GPIO UNCLAIMED) function i2s0 group i2s0-idle
......
```

It can be seen that gpio1-16 and gpio1-17 are set as UART pins. After calling `iomux_exchange(1)`, the command is input to view:

```
console:/ # cat sys/kernel/debug/pinctrl/pinctrl-rockchip-pinctrl/pinmux-pins
......
pin 44 (gpio1-12): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 45 (gpio1-13): fde60000.dp gpio1:45 function dp group dp1-hpd
pin 46 (gpio1-14): 5-001a (GPIO UNCLAIMED) function mipi group mipim0-camera1-clk
pin 47 (gpio1-15): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 48 (gpio1-16): iomux-exchange (GPIO UNCLAIMED) function uart3-gpio group uart3m0-gpio
pin 49 (gpio1-17): iomux-exchange (GPIO UNCLAIMED) function uart3-gpio group uart3m0-gpio
pin 50 (gpio1-18): 7-0011 (GPIO UNCLAIMED) function i2s0 group i2s0-mclk
pin 51 (gpio1-19): fe470000.i2s (GPIO UNCLAIMED) function i2s0 group i2s0-idle
pin 52 (gpio1-20): (MUX UNCLAIMED) gpio1:52
pin 53 (gpio1-21): fe470000.i2s (GPIO UNCLAIMED) function i2s0 group i2s0-idle
......
```

It can be seen that gpio1-16 and gpio1-17 have been successfully repurposed as GPIO functions.

3. GPIO Development in U-Boot

Users have the need to operate GPIO in U-Boot, which can be done by directly reading and writing to the register address using `readl` and `writel`. However, this method is not portable. Here is a method to operate GPIO through U-Boot drivers. Taking the RK3588 Android12 platform as an example, the specific U-Boot DTS configuration and driver code are as follows.

3.1 U-Boot LED Driver Development

DTS Configuration

U-Boot has its own DTS files, which automatically generate the corresponding DTB files during compilation, appended at the end of u-boot.bin. The DTS configuration for developing GPIO in U-Boot is the same as in the Kernel, but the `u-boot,dm-spl` or `u-boot,dm-pre-reloc` property needs to be added in the node.

```
/{
    leds {
        compatible = "gpio-leds";
        status = "okay";
        u-boot,dm-spl;

        blue-led {
            u-boot,dm-spl;
            gpios = <&gpio2 RK_PA1 GPIO_ACTIVE_LOW>;
            label = "blue_full";
            default-state = "on";
        };
```

```
        green-led {
            u-boot,dm-spl;
            gpios = <&gpio2 RK_PA0 GPIO_ACTIVE_LOW>;
            label = "green_led";
            default-state = "on";
        };
    };
};
```

Note: In the DTS file, the configuration of "gpio-leds" is generally placed in the Kernel, and the "label" configuration option needs to be added, otherwise, the led_gpio.c driver will bind incorrectly. The following patch can print the "label" configuration option:

```
diff --git a/drivers/led/led_gpio.c b/drivers/led/led_gpio.c
index 56702a1417..18062f2ac1 100644
--- a/drivers/led/led_gpio.c
+++ b/drivers/led/led_gpio.c
@@ -112,6 +112,7 @@ static int led_gpio_bind(struct udevice *parent)
                const char *label;

                label = ofnode_read_string(node, "label");
+               printf("%s: label %s\n", __func__, label);
                if (!label) {
                        debug("%s: node %s has no label\n", __func__,
                              ofnode_get_name(node));
```

After compiling U-Boot, users can check the DTB content using the `fdtdump` command:

```
fdtdump ./u-boot.dtb | less
```

User Driver Code

This section mainly introduces the LED driver in U-Boot. First, enable the configuration:

```
/u-boot/configs/rk3588_defconfig
CONFIG_LED=y
CONFIG_LED_GPIO=y
```

The specific framework code reference is as follows:

```
u-boot/drivers/led/led-uclass.c // Default compilation
u-boot/drivers/led/led_gpio.c // Supports compatible = "gpio-leds"
```

Related code interfaces

```
// Get led device
int led_get_by_label(const char *label, struct udevice **devp);
// Set/Get led state
int led_set_state(struct udevice *dev, enum led_state_t state);
enum led_state_t led_get_state(struct udevice *dev);
// Ignore, not implemented in the underlying driver currently
int led_set_period(struct udevice *dev, int period_ms);
```

Users only need to call `led_get_by_label` to achieve LED lighting. The reference code is as follows, adding `led_get_by_label` to `board_late_init` execution, after the system starts, you can observe the LED lights on gpio2a0 and gpio2a1 pins.

```
/u-boot/arch/arm/mach-rockchip/board.c
#include <led.h>

int board_late_init(void)
{
    struct udevice *led_dev;
    ...
    led_get_by_label("blue_full", &led_dev);
    led_set_state(led_dev, LEDST_ON);
    ...
}
```

U-Boot differs from the Kernel driver; although it introduces the device-driver development model, it does not automatically initiate the probe of registered device-drivers during the initialization phase like the Kernel. The driver's probe must be actively called by the user. That is, after the user actively calls `led_get_by_label`, the LED driver probe function `led_gpio_probe` will be executed. The core call of `led_get_by_label` is to execute the LED driver's probe through `device_probe`. And you can set the LED light on/off state with `led_set_state`.

Referencing the LED driver code, users can design their own U-Boot GPIO drivers. Obtaining user device driver interfaces is as follows:

```
int uclass_get_device(enum uclass_id id, int index, struct udevice **devp);
int uclass_get_device_by_name(enum uclass_id id, const char *name,
struct udevice **devp);
int uclass_get_device_by_seq(enum uclass_id id, int seq, struct udevice **devp);
int uclass_get_device_by_of_offset(enum uclass_id id, int node, struct udevice
**devp);
int uclass_get_device_by_ofnode(enum uclass_id id, ofnode node, struct udevice
**devp);
int uclass_get_device_by_phandle_id(enum uclass_id id,
int phandle_id, struct udevice **devp);
int uclass_get_device_by_phandle(enum uclass_id id,
struct udevice *parent, struct udevice **devp);
int uclass_get_device_by_driver(enum uclass_id id,
const struct driver *drv, struct udevice
**devp);
int uclass_get_device_tail(struct udevice *dev, int ret, struct udevice **devp);
......
```

GPIO operation interface code is as follows:

```
// Request/Release GPIO
int gpio_request_by_name(struct udevice *dev, const char *list_name,
int index, struct gpio_desc *desc, int flags);
int gpio_request_by_name_nodev(ofnode node, const char *list_name, int index,
struct gpio_desc *desc, int flags);
int gpio_request_list_by_name(struct udevice *dev, const char *list_name,
struct gpio_desc *desc_list, int max_count, int
flags);
int gpio_request_list_by_name_nodev(ofnode node, const char *list_name,
struct gpio_desc *desc_list, int max_count,
int flags);
int dm_gpio_free(struct udevice *dev, struct gpio_desc *desc)
// Set GPIO direction. @flags: GPIOD_IS_OUT (output) and GPIOD_IS_IN (input)
int dm_gpio_set_dir_flags(struct gpio_desc *desc, ulong flags);
// Set/Get GPIO level
int dm_gpio_get_value(const struct gpio_desc *desc)
int dm_gpio_set_value(const struct gpio_desc *desc, int value)
```

`udevice` is obtained through `uclass_id`, and the LED driver registration is as follows:

```
U_BOOT_DRIVER(led_gpio) = {
  .name  = "gpio_led",
  .id = UCLASS_LED,
  .of_match = led_gpio_ids,
  .ops   = &gpio_led_ops,
  .priv_auto_alloc_size = sizeof(struct led_gpio_priv),
  .bind  = led_gpio_bind,
  .probe  = led_gpio_probe,
  .remove = led_gpio_remove,
};
```

`UCLASS_LED` is defined in u-boot/include/dm/uclass-id.h.

3.2 U-Boot GPIO Driver Development

The LED driver mentioned above has already been implemented in the U-Boot driver; users only need to configure the DTS and call the driver interface. Below is a demo for the development of the U-Boot GPIO driver, with the following configurations enabled:

```
/u-boot/configs/rk3588_defconfig
CONFIG_DM_GPIO=y
CONFIG_GPIO_HOG=y
CONFIG_ARCH_ROCKCHIP=y
CONFIG_SPL_GPIO_SUPPORT=y
```

DTS Configuration

Taking the operation of gpio3b7 as an example, the U-Boot DTS configuration is as follows:

```
gpio_uboot_demo {
    u-boot,dm-pre-reloc;
    compatible="gpio_uboot_demo";
    enable-gpios = <&gpio3 RK_PB7 GPIO_ACTIVE_HIGH>;
    status = "okay";
};

&gpio3 {
    u-boot,dm-pre-reloc;
    status = "okay";
};
```

User Driver Code

```
/*
 * Copyright (c) 2015 Google, Inc
 * Written by Simon Glass <sjg@chromium.org>
 *
 * SPDX-License-Identifier:    GPL-2.0+
 */

/* The following code is based on the RK3588 Android 12 platform */
#include <common.h>
#include <dm.h>
#include <errno.h>
#include <asm/gpio.h>
#include <dm/lists.h>
#include <dm/ofnode.h>

static int gpio_uboot_demo_probe(struct udevice *dev)
{
    struct gpio_desc gpio_uboot;
    int ret;

    printf("----gpio_uboot_demo_probe----\n");
    ret = gpio_request_by_name(dev, "enable-gpios", 0, &gpio_uboot, GPIOD_IS_OUT);
    if(ret)
        printf("can not get enable gpio\n");

    dm_gpio_set_value(&gpio_uboot, 1);
    return 0;
}

static int gpio_uboot_demo_remove(struct udevice *dev)
{
    printf("----gpio_uboot_demo_remove----\n");
    return 0;
}

static const struct udevice_id gpio_uboot_demo_ids[] = {
```

```
    { .compatible = "gpio_uboot_demo" },
    { }
};

U_BOOT_DRIVER(gpio_uboot_demo) = {
    .name        = "gpio_uboot_demo",
    .id            = UCLASS_GPIO,
    .of_match    = gpio_uboot_demo_ids,
    .probe        = gpio_uboot_demo_probe,
    .remove        = gpio_uboot_demo_remove,
};
```

The user driver obtains the gpio3b7 pin through `gpio_request_by_name(dev, "enable-gpios", 0, &gpio_uboot, GPIOD_IS_OUT)` and sets it as an output. The GPIO probe function needs to be actively called by the user through the relevant interface, as follows:

```
/u-boot/arch/arm/mach-rockchip/board.c
int board_late_init(void)
{
    struct udevice *dev;
    ...
    uclass_get_device_by_driver(UCLASS_GPIO, DM_GET_DRIVER(gpio_uboot_demo), &dev);
    ...
}
```

In `board_late_init`, calling `uclass_get_device_by_driver` will eventually call the aforementioned driver's probe, and `dm_gpio_set_value(&gpio_uboot, 1)` will output a high level.

4. User-space GPIO Development

There are various ways to develop GPIO in user space, and this example will illustrate using the common echo command. The Linux kernel provides a set of interfaces for debugging GPIO in user space, located in the `/sys/class/gpio/` directory:

```
console:/sys/class/gpio# ls
export     gpiochip128  gpiochip509  gpiochip96
gpiochip0  gpiochip32   gpiochip64   unexport
```

The `export` file is used to notify the system to export the GPIO pin number that needs to be controlled. `unexport` is used to notify the system to cancel the export. `gpiochipN` stores information about the GPIO registers in the system, including the base number of the control pin for each register, the register name, and the total number of pins.

As an example, let's export pin gpio1b1 for operation. First, calculate the pin number: pin number = base number of the control pin register + bit position of the control pin register. gpio1b1 = 32 + 9 = 41. Export gpio1b1:

```
console:/sys/class/gpio# echo 41 > export
```

Set input/output, "in" for input, "out" for output:

```
console:/sys/class/gpio/gpio41# echo out > direction
```

Check direction:

```
console:/sys/class/gpio/gpio41# cat direction
```

Set output:

```
console:/sys/class/gpio/gpio41# echo 1 > value
```

Check input/output:

```
console:/sys/class/gpio/gpio41# cat value
```

Cancel export:

```
console:/sys/class/gpio# echo 41 > unexport
```

The above operations require ensuring that IO multiplexing is set to GPIO functionality to perform normal GPIO operations. If it is multiplexed for other functions, the GPIO output and input cannot be operated. It is necessary to check the IO multiplexing situation in advance. In addition to using IO tools to view the register to see IO multiplexing and configuration, you can also use the following command to view all pin multiplexing and configuration:

```
console:/ # cat sys/kernel/debug/pinctrl/pinctrl-rockchip-pinctrl/pinmux-pins
Pinmux settings per pin
Format: pin (name): mux_owner gpio_owner hog?
pin 0 (gpio0-0): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 1 (gpio0-1): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 2 (gpio0-2): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 3 (gpio0-3): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 4 (gpio0-4): (MUX UNCLAIMED) gpio0:4
pin 5 (gpio0-5): (MUX UNCLAIMED) gpio0:5
pin 6 (gpio0-6): ff420030.pwm (GPIO UNCLAIMED) function pwm3a group pwm3a-pin
pin 7 (gpio0-7): fe320000.dwmmc (GPIO UNCLAIMED) function sdmmc group sdmcc-cd
pin 8 (gpio0-8): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 9 (gpio0-9): (MUX UNCLAIMED) gpio0:9
pin 10 (gpio0-10): (MUX UNCLAIMED) gpio0:10
......

console:/ # cat sys/kernel/debug/gpio
gpiochip0: GPIOs 0-31, parent: platform/pinctrl, gpio0:
 gpio-1   (                    |vcc3v0_sd           ) out hi
 gpio-4   (                    |bt_default_wake_host) in  lo
 gpio-5   (                    |GPIO Key Power      ) in  hi
```

```
 gpio-9   (                    |bt_default_reset    ) out lo
 gpio-10  (                    |reset               ) out hi
 gpio-11  (                    |spk-con             ) out lo

gpiochip1: GPIOs 32-63, parent: platform/pinctrl, gpio1:
 gpio-34  (                    |int-n               ) in  hi
 gpio-45  (                    |enable              ) out hi

gpiochip2: GPIOs 64-95, parent: platform/pinctrl, gpio2:
 gpio-64  (                    |vbus-5v             ) out lo
 gpio-83  (                    |bt_default_rts      ) in  hi
 gpio-90  (                    |bt_default_wake     ) in  lo

gpiochip3: GPIOs 96-127, parent: platform/pinctrl, gpio3:
 gpio-111 (                    |mdio-reset          ) out hi

gpiochip4: GPIOs 128-159, parent: platform/pinctrl, gpio4:
 gpio-150 (                    |?                   ) out hi
 gpio-153 (                    |vcc5v0_host         ) out hi
 gpio-157 (                    |enable              ) out hi
 gpio-158 (                    |vcc_lcd             ) out hi
```

5. IO Sleep Wakeup Configuration

The current chip-level sleep and wakeup operations are primarily handled in ATF, and this part of the code is not open. To meet the needs of different products, the system can enter different low-power modes during sleep by configuring DTS. Different projects have different requirements for wake-up sources, and the corresponding wake-up sources can be enabled in DTS.

The general process for system standby typically includes the following operations: closing power domains, module IPs, clocks, PLLs, entering self-refresh for DDR, switching the system bus to a low-speed clock (24M or 32K), powering off vdd_arm/vdd_log, and configuring wake-up sources. To meet the standby mode requirements of different products, configurations are currently passed to trust via DTS nodes during the boot stage. Taking the RK3588 platform as an example:

```
arch/arm64/boot/dts/rockchip/rk3588s.dtsi
rockchip_suspend: rockchip-suspend {
        compatible = "rockchip,pm-rk3588";
        status = "disabled";
        // Set the sleep log switch. 0: printing is disabled, 1: printing is enabled
        rockchip,sleep-debug-en = <0>;
         // Sleep configuration
        rockchip,sleep-mode-config = <
                (0
                | RKPM_SLP_ARMOFF_LOGOFF //Enter deep sleep and power off vdd_arm and vdd_log
                | RKPM_SLP_PMU_PMUALIVE_32K //Use 32K clock source as system clock when
entering deep sleep
                | RKPM_SLP_PMU_DIS_OSC //Turn off 24M crystal, can be enabled when in the
lowest power mode
```

```
                | RKPM_SLP_32K_EXT //Whether to use an external 32K clock source during sleep
                )
        >;
        // Wakeup source configuration
        rockchip,wakeup-config = <
                (0
                | RKPM_GPIO_WKUP_EN //GPIO0 wake-up
                )
        >;
};
```

5.1 Configuring GPIO Wakeup

Taking the RK3588 as an example, the pins of GPIO0 group are configured to be powered on by default during sleep, so users do not need to configure them again. If you need to configure other GPIOs besides GPIO0 as a wake-up source, the following configurations are required:

- Configure this GPIO to not power off during sleep

```
vdd_log_s0: DCDC_REG3 {
    regulator-always-on;
    regulator-boot-on;
    regulator-min-microvolt = <675000>;
    regulator-max-microvolt = <750000>;
    regulator-ramp-delay = <12500>;
    regulator-name = "vdd_log_s0";
    regulator-state-mem {
    // regulator-off-in-suspend;
    regulator-on-in-suspend;
    regulator-suspend-microvolt = <750000>;
    };
};
```

`regulator-off-in-suspend` indicates that the power supply for this route should be turned off during sleep; if it needs to be turned on during sleep, change it to `regulator-on-in-suspend`.

- Configure the rockchip_suspend node

```
&rockchip_suspend {
        status = "okay";
        rockchip,sleep-debug-en = <1>;
        rockchip,sleep-mode-config = <
                (0
                 // | RKPM_SLP_ARMOFF_LOGOFF
                | RKPM_SLP_ARMOFF_DDRPD
                | RKPM_SLP_PMU_PMUALIVE_32K
                | RKPM_SLP_PMU_DIS_OSC
                | RKPM_SLP_32K_EXT
                )
        >;
```

```
        rockchip,wakeup-config = <
                (0
                | RKPM_GPIO_WKUP_EN
                | RKPM_CPU0_WKUP_EN
                )
        >;
};
```

In the rockchip_suspend node, `RKPM_SLP_ARMOFF_LOGOFF` indicates that vdd_arm and vdd_log are powered off; change it to `RKPM_SLP_ARMOFF_DDRPD` to keep vdd_log powered on. The rockchip_suspend node also needs to be configured with `RKPM_CPU0_WKUP_EN`.

Note that not all GPIOs can be used as wake-up sources. For specifics, please refer to the TRM document PMU_WAKEUP_INT_CON register; many platforms only support GPIO0 group IO ports as wake-up sources.

5.2 Configuring GPIO Levels During Sleep

Using the RK3399 platform as an example:

```
&rockchip_suspend {
    status = "okay";
+    rockchip,power-ctrl =
+      <&gpio1 17 GPIO_ACTIVE_HIGH>, // Configure GPIO1_C1 to high level during sleep
+      <&gpio1 14 GPIO_ACTIVE_HIGH>; // Configure GPIO1_B6 to high level during sleep
```

rockchip,power-ctrl can only select GPIOs on PMUIO1/2, it is best to place this IO on gpio0, otherwise the IO status cannot be maintained after sleep. The new sleep framework will restore IOs in non-pmu controlled power domains to their default state during sleep.

6. GPIO Expansion Chips

This section introduces the basic configuration and calling methods of GPIO expansion chips. The following uses PCA9555 as an example to illustrate (PCA9555 is a 16-bit general-purpose input/output GPIO expander with interrupts and weak pull-up resistors, suitable for I2c bus/SMBus applications).

DTS Configuration

```
&i2c6 {
    status = "okay";

    pca9555_20: pca9555_20@20 {
        compatible = "nxp,pca9555";
        reg = <0x20>;
    };

    imx586: imx586@1a {
```

```
        compatible = "sony,imx586";
        status = "okay";
        reset-gpio-index = <503>;
        ......
    };
};
```

The `reset-gpio-index` is the index number of the gpio, which can be confirmed through the command `cat /sys/kernel/debug/gpio`.

Driver Invocation

```
ret |= of_property_read_u32(node, "reset-gpio-index",
        &test->reset_gpio_index);

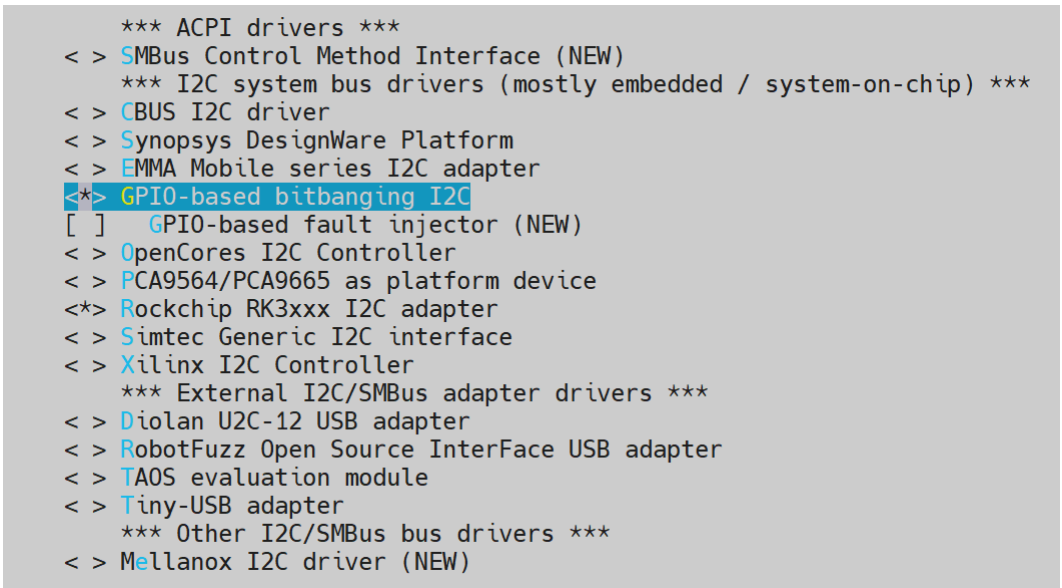
gpio_to_desc(test->reset_gpio_index);
```

7. GPIO Simulating I2C

This section introduces the basic configuration of GPIO simulating I2C. The corresponding driver link is: kernel/drivers/i2c/busses/i2c-gpio.c

config configuration

In defconfig, set CONFIG_I2C_GPIO=y, or select GPIO-based bitbanging I2C in menuconfig.



The corresponding options are:

```
Device Drivers->
    I2C support  --->
        I2C Hardware Bus support  --->
            <*> GPIO-based bitbanging I2C
```

dts configuration

```
i2c9: i2c@9 {
    compatible = "i2c-gpio";
    pinctrl-names = "default";
    pinctrl-0 = <&i2c9_xfer>;
    scl-gpios = <&gpio3 RK_PB3 GPIO_ACTIVE_HIGH>; //scl
    sda-gpios = <&gpio3 RK_PB4 GPIO_ACTIVE_HIGH>; //sda
    i2c-gpio,delay-us = <2>;    /* ~100 kHz */
    #address-cells = <1>;
    #size-cells = <0>;
};

&pinctrl {
    i2c9 {
        /omit-if-no-ref/
        i2c9_xfer: i2c9-xfer {
            rockchip,pins =
                /* i2c9_scl */
                <3 RK_PB3 0 &pcfg_pull_none_smt>,
                /* i2c9_sda */
                <3 RK_PB4 0 &pcfg_pull_none_smt>;
        };
    };
};
```

8. GPIO Debug Tools

This section introduces the debugging tools for GPIO.

8.1 IO Tools

Rockchip platform provides IO tools for directly reading and setting the values of registers, facilitating user debugging.

8.1.1 Kernel Phase IO Tools

For Android 10 and above due to security standards, manual configuration is required during debugging. For example, on the RK3399 Android 12 platform, the following configurations need to be enabled to use IO instructions:

```
kernel-5.10/kernel/configs/android-11.config
CONFIG_DEVMEM=y
```

A detailed explanation of the command parameters included in the IO tool can be listed by entering "io" in the serial terminal or adb.

```
console:/# io
Raw memory i/o utility - $Revision: 1.5 $

io -v -1|2|4 -r|w [-l <len>] [-f <file>] <addr> [<value>]

    -v         Verbose, asks for confirmation
    -1|2|4     Sets memory access size in bytes (default byte)
    -l <len>   Length in bytes of area to access (defaults to
               one access, or whole file length)
    -r|w       Read from or Write to memory (default read)
    -f <file>  File to write on memory read, or
               to read on memory write
    <addr>     The memory address to access
    <val>      The value to write (implies -w)

Examples:
    io 0x1000                  Reads one byte from 0x1000
    io 0x1000 0x12             Writes 0x12 to location 0x1000
    io -2 -l 8 0x1000          Reads 8 words from 0x1000
    io -r -f dmp -l 100 200    Reads 100 bytes from addr 200 to file
    io -w -f img 0x10000       Writes the whole of file to memory

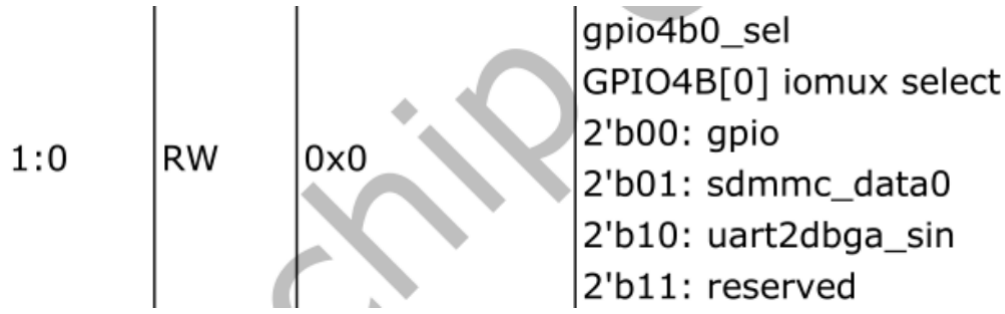
Note access size (-1|2|4) does not apply to file based accesses.
```

IO Tool Query GPIO Muxing

Using the RK3399 platform as an example, users can determine the existence of IO multiplexing by consulting the RK3399 main control chip detailed specification book TRM to query the registers. For instance, if you want to query the gpio4b0 pin, the schematic diagram shows the following multiplexing states:

```
GPIO4_B0/SDMMC0_D0/UART2A_RX_u
```

By searching for gpio4b0 in the RK3399 TRM, you can find the register description as follows:



1:0	RW	0x0	gpio4b0_sel GPIO4B[0] iomux select 2'b00: gpio 2'b01: sdmmc_data0 2'b10: uart2dbga_sin 2'b11: reserved

By determining the base address name of the register as "GRF" with an offset of 0x0e024 from the TRM documentation:

GRF_GPIO4B_IOMUX

Address: Operational Base + offset (0x0e024)

GPIO4B iomux control

By searching for address mapping in the datasheet and finding the base address of GRF:

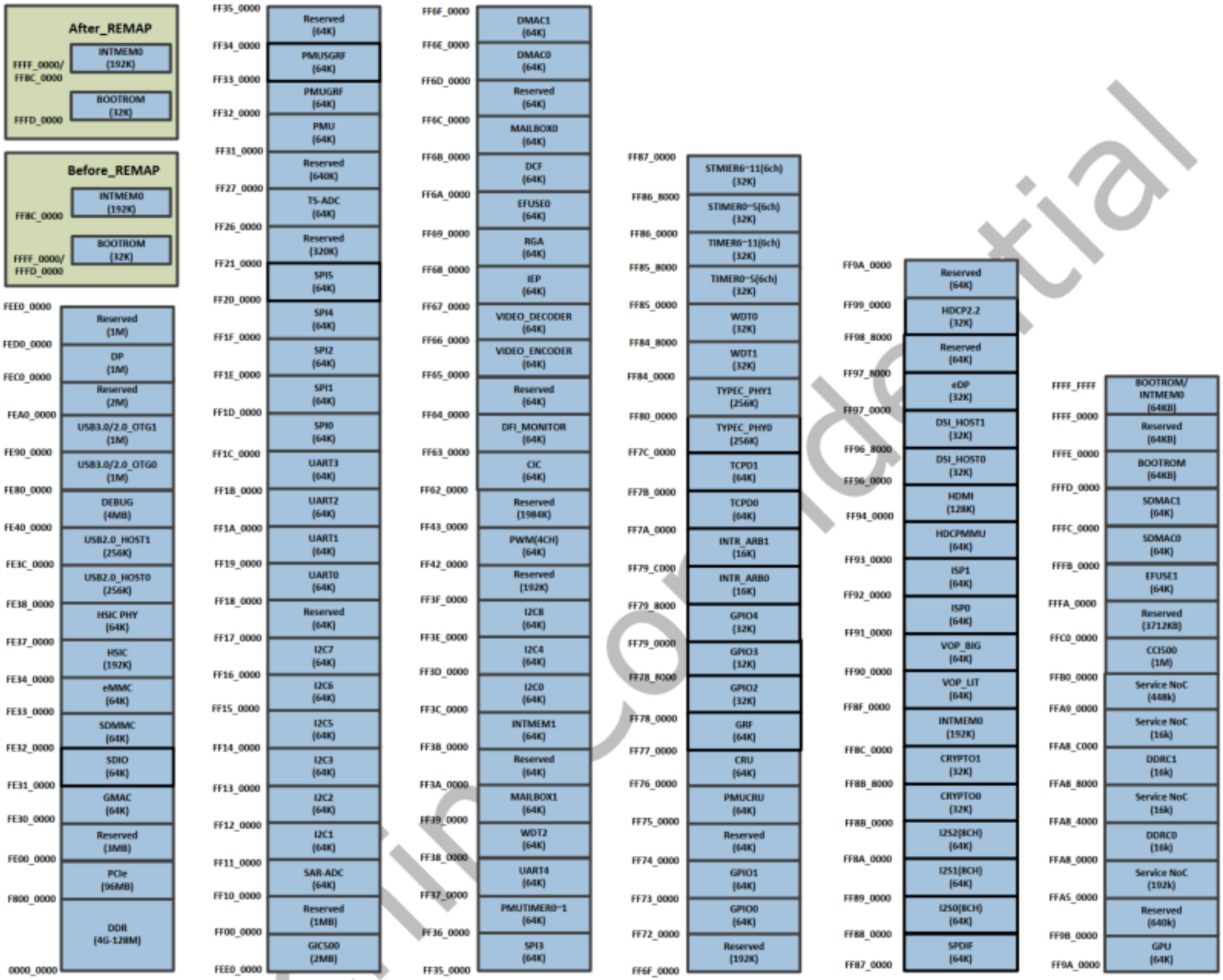
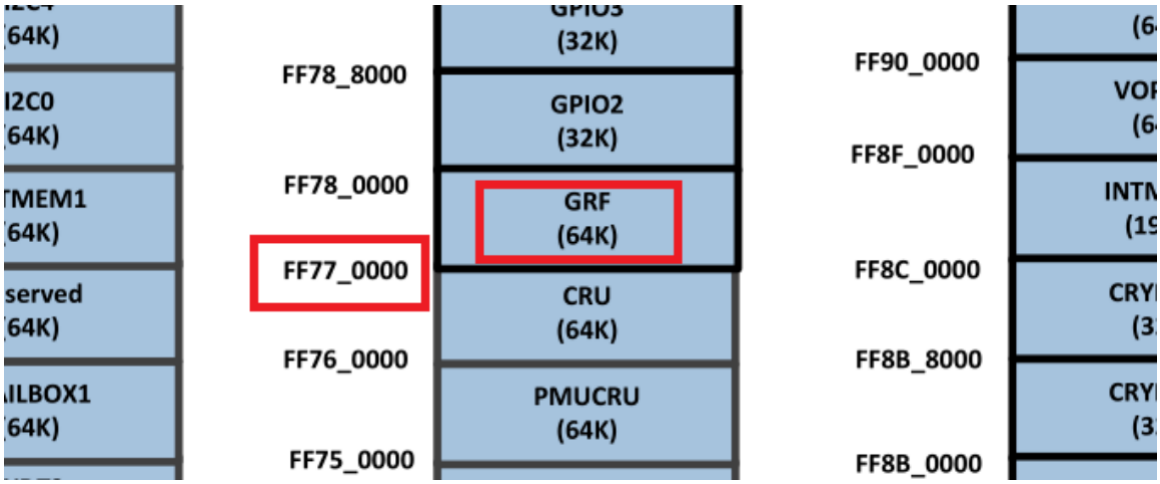


Fig. 2-1 RK3399 Address Mapping



After determining the base address of GRF, adding the offset gives the multiplexing register address for gpio4b0 as `0xff77e024`. Enter in the serial port or adb:

```
console:/ # io -4 -l 0x100 0xff77e024
ff77e024: 00000555 0000855f 00000000 00000000
ff77e034: 00000000 00000000 00000000 000002a5
ff77e044: 00005450 0000ff03 00000143 00000000
ff77e054: 00004010 00000001 00000000 0000aa80
ff77e064: 00000455 00002040 0000aaa1 00000000
ff77e074: 00000000 00000000 00000000 00000000
ff77e084: 00000000 00000000 00000000 00000000
ff77e094: 00000000 00000000 00000000 00000000
ff77e0a4: 000000ff 00000000 00000000 00000000
ff77e0b4: 00000000 00000000 00000000 00000000
ff77e0c4: 00000000 00000000 00000000 00000000
ff77e0d4: 00000000 00000000 00000000 00000000
ff77e0e4: 00000000 00000000 00000000 00000000
ff77e0f4: 00000000 00000000 00000000 00000000
ff77e104: 00000000 00000000 00000000 0000b01b
ff77e114: 00000001 00003000 00000000 00000018
```

The result of the first line is:

```
ff77e024: 00000555 0000855f 00000000 00000000
```

The four values correspond to the addresses:

```
0xff77e024, 0xff77e028, 0xff77e02c, 0xff77e030
```

Correspondence (hexadecimal):

```
0xff77e024: 00000555
```

0x555 converted to binary, padded to 32 bits:

```
0000 0000 0000 0000 0000 0101 0101 0101
```

The result after conversion, from right to left, is from low bit to high bit, that is, the bit on the far right is the 0th bit, and the bit on the far left is the 31st bit. Then, corresponding to the previously queried register `GRF_GPIO4B_IOMUX`, you can check the results one by one.

1:0	RW	0x0	gpio4b0_sel GPIO4B[0] iomux select 2'b00: gpio 2'b01: sdmmc_data0 2'b10: uart2dbga_sin 2'b11: reserved

By viewing the above serial output results converted to binary, the [1:0] bit is 01, corresponding to the `sdmmc_data0` function of this register. Thus, it is confirmed that the current multiplexing situation of the gpio4b0 IO pin. If this is not the state the user needs to set, then look in the code to find where it is reused.

How to find the reuse in the code? For example, if the gpio4b0 pin is reused as a GPIO function, to pull high or low, but the above query result shows `sdmmc_data0` function. At this time, we should first go to the dts to find the sdmmc node, and confirm whether the pinctrl attribute has reused this gpio repeatedly.

IO Tool Write Register Operation

If you want to temporarily write a register for experimentation through the IO command, you can use `io -w` to write.

For example, the value of the register has been read out through the command `io -4 -r 0xff77e024`. At this time, if you want to write 1 to the 0th bit of the `0xff77e024` register, you can operate as follows:

Note: Why do you have to write 1 to the 16th bit of the hexadecimal value behind the register address? Because the 16th to 31st bits of this register are write enable bits, which are 0 by default, that is, not writable. Since you want to write 1 to the 0th bit, the write enable bit corresponding to the 0th bit, the 16th bit, must also be set to 1 to write, which is determined by the register description:

GRF_GPIO4B_IOMUX
Address: Operational Base + offset (0x0e024)
GPIO4B iomux control

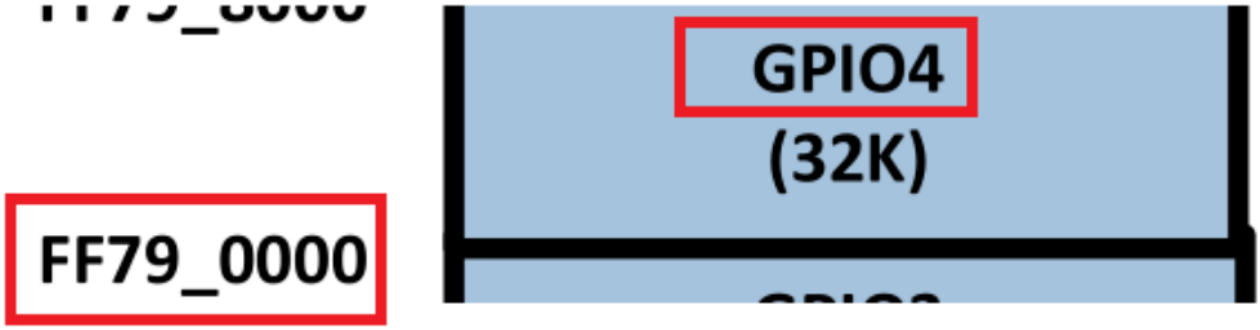
Bit	Attr	Reset Value	Description
31:16	RW	0x0000	write_enable bit0~15 write enable When bit 16=1, bit 0 can be written by software . When bit 16=0, bit 0 cannot be written by software; When bit 17=1, bit 1 can be written by software . When bit 17=0, bit 1 cannot be written by software; ..... When bit 31=1, bit 15 can be written by software . When bit 31=0, bit 15 cannot be written by software;

Rockchip Co

It should be noted that the register values written through the IO tool will not be retained after reboot.

IO Tool View GPIO Register

If you want to confirm whether the current GPIO is input or output, output high or output low, etc., you can also analyze by reading the register. Taking gpio4b0 as an example, by searching "ADDRESS MAPPING" in the RK3399 TRM, you can find the base address of gpio4:



By using the io command to query the status of gpio4, as follows:

```
console:/ # io -4 -l 0x100 0xFF790000
ff790000: 42000000 6a400000 00000000 00000000
ff790010: 00000000 00000000 00000000 00000000
ff790020: 00000000 00000000 00000000 00000000
ff790030: ffffffff ffffffff 00000000 00000000
ff790040: 00000000 9434d0f9 00000000 00000000
ff790050: 438b2f06 00000000 00000000 00000000
ff790060: 00000000 00000000 00000000 3230372a
ff790070: 00000000 00000000 00000000 00000000
ff790080: 42000000 6a400000 00000000 00000000
ff790090: 00000000 00000000 00000000 00000000
ff7900a0: 00000000 00000000 00000000 00000000
ff7900b0: ffffffff ffffffff 00000000 00000000
ff7900c0: 00000000 9434d0f9 00000000 00000000
ff7900d0: 438b2f06 00000000 00000000 00000000
ff7900e0: 00000000 00000000 00000000 3230372a
ff7900f0: 00000000 00000000 00000000 00000000
```

According to the GPIO chapter register introduction in the RK3399 TRM, for example, to see the status of the gpio4b0 pin, based on the register description:

10.4.1 Registers Summary

Name	Offset	Size	Reset Value	Description
GPIO_SWPORTA_DR	0x0000	W	0x00000000	Port A data register
GPIO_SWPORTA_DDR	0x0004	W	0x00000000	Port A data direction register
GPIO_INTEN	0x0030	W	0x00000000	Interrupt enable register
GPIO_INTMASK	0x0034	W	0x00000000	Interrupt mask register
GPIO_INTTYPE_LEVEL	0x0038	W	0x00000000	Interrupt level register
GPIO_INT_POLARITY	0x003c	W	0x00000000	Interrupt polarity register
GPIO_INT_STATUS	0x0040	W	0x00000000	Interrupt status of port A
GPIO_INT_RAWSTATUS	0x0044	W	0x00000000	Raw Interrupt status of port A
GPIO_DEBOUNCE	0x0048	W	0x00000000	Debounce enable register
GPIO_PORTA_EOI	0x004c	W	0x00000000	Port A clear interrupt register
GPIO_EXT_PORTA	0x0050	W	0x00000000	Port A external port register
GPIO_LS_SYNC	0x0060	W	0x00000000	Level_sensitive synchronization enable register

*Notes:Size:**B**- Byte (8 bits) access, **HW**- Half WORD (16 bits) access, **W**-WORD (32 bits) access*

Rockchip Confi

0x4 offset is the input/output status register, and 0x0 offset is the data register. View the input/output status register `0xff790004`, the result is 0x6a400000, the result corresponding bit is:

gpio4d	gpio4c	gpio4b	gpio4a
0x6a	0x40	0x00	0x00

And each two hexadecimal digits converted to binary values from right to left correspond to 0 to 7 feet, for example, 0x6a binary is: `0110 1010`, then the binary from right to left corresponds to the status of gpio4d0 to gpio4d7. According to the register description:

RK3399 TRM

Address: Operational Base + offset (0x0004)
Port A data direction register

Bit	Attr	Reset Value	Description
31:0	RW	0x00000000	gpio_swporta_ddr Values written to this register independently control the direction of the corresponding data bit in Port A. 0: Input (default) 1: Output

So at this time, gpio4b0 is in the input state. If it is an output state at this time, you can check the value of `0xFF790000` to confirm whether the output is high or low. If it is an input state, you can check the input voltage status by viewing `0xFF790050` (GPIO_EXT_PORTA).

8.1.2 U-Boot Phase IO Tools

During the U-Boot boot phase, press and hold the `ctrl + c` key combination to forcibly enter the command line. The U-Boot command line mode provides many commands, and typing `?` will list all the currently supported commands:

```
=> ?
?       - alias for 'help'
android_print_hdr- print android image header
base    - print or set address offset
bdinfo  - print Board Info structure
bidram_dump- Dump bidram layout
boot    - boot default, i.e., run 'bootcmd'
……
iomem   - Show iomem data by device compatible(high priority) or node name
……
md      - memory display
……
```

For earlier versions, please recompile and download U-Boot with the following configuration:

- To use IO commands, the following macro must be enabled:

```
diff --git a/include/configs/rk_default_config.h
b/include/configs/rk_default_config.h
index 9852917d4f..305de26347 100755
--- a/include/configs/rk_default_config.h
+++ b/include/configs/rk_default_config.h
@@ -304,7 +304,7 @@
/* rk io command tool */
-#undef CONFIG_RK_IO_TOOL
+#define CONFIG_RK_IO_TOOL
```

U-Boot can also use the md command

- To use the md command, add the following macro:

```
diff --git a/include/configs/rk33plat.h b/include/configs/rk33plat.h
index 0906c0c091..3f94e9eabe 100755
--- a/include/configs/rk33plat.h
+++ b/include/configs/rk33plat.h
@@ -140,6 +140,7 @@
#define CONFIG_RK_GPIO_EXT_FUNC
#define CONFIG_CHARGE_LED
#define CONFIG_POWER_FUSB302
+ #define CONFIG_CMD_MEMORY
#endif
#if defined(CONFIG_RKCHIP_RK322XH)
```

- To set the boot delay:

```
diff --git a/common/autoboot.c b/common/autoboot.c
index c27cc2c751..fe28adb2fd 100644
--- a/common/autoboot.c
+++ b/common/autoboot.c
@@ -146,6 +146,7 @@ static int abortboot_normal(int bootdelay)
{
int abort = 0;
unsigned long ts;
+ bootdelay = 3; // Add time here as needed, unit: seconds
#ifdef CONFIG_MENUPROMPT
printf(CONFIG_MENUPROMPT);
```

The default configuration for the Rockchip platform is already enabled, and it is recommended to use the md command to read and write registers. The operation of the md command is as follows:

```
// Read operation
md - memory display
Usage: md [.b, .w, .l, .q] address [# of objects]
// Write operation
mw - memory write (fill)
Usage: mw [.b, .w, .l, .q] address value [count]
```

For reading operations, refer to the following: display continuous 0x10 data starting from address `0x76000000`.

```
=> md.l 0x76000000 0x10
76000000: fffffffe ffffffff ffffffff ffffffff ................
76000010: fffffffdf ffffffff feffffff ffffffff ................
76000020: ffffffff ffffffff ffffffff ffffffff ................
76000030: ffffffff ffffffff ffffffff ffffffff ................
```

For writing operations, refer to the following: assign the value 0x1234 to address `0x76000000`.

```
=> mw.l 0x76000000 0xffff1234 // High 16 bits have a mask
=> md.l 0x76000000 0x10 // Read back
76000000: ffff1234 ffffffff ffffffff ffffffff ................
76000010: fffffffdf ffffffff feffffff ffffffff ................
76000020: ffffffff ffffffff ffffffff ffffffff ................
76000030: ffffffff ffffffff ffffffff ffffffff ................
```

8.2 IOMUX Tool

The iomux is a binary file compiled by gcc, which can acquire and set the GPIO pin through the ioctl call to the rockchip-pinctrl device. It is not enabled by default and requires setting CONFIG_ROCKCHIP_IOMUX=y in the defconfig.

Source code for the iomux tool:

```
kernel/tools/testing/selftests/rkpinctrl/iomux.c
```

Compilation method:

```
gcc kernel/tools/testing/selftests/rkpinctrl/iomux.c -o iomux
```

Usage:

Example: Set GPIO0_B7 to func1

```
[root@RK3588:/]# iomux 0 15 1
```

Example: Get GPIO0_B7 iomux

```
[root@RK3588:/]# iomux 0 15
mux get (GPIO0-15) = 1
```

8.3 Pinconfig Tool

The PinConfig tool provides a unified IO configuration and IO verification function for hardware and software personnel. It currently supports the chips `RK3588|RK3566|RK3568|RV1126|RV1109`.

For detailed usage, please refer to the document "*PinConfig_UserManual.pdf*" within the tool.

8.4 PinDebug Tool

The PinDebug tool is a debugging tool specifically designed for pins, capable of querying the iomux of pins and configuring pin properties. It currently supports chips `RK3588|RK3566|RK3568|RV1126|RV1109|RV1106|RV1103|RK3528`.

For detailed usage instructions, please refer to the document "*PinDebug_UserManual.pdf*" within the tool.

9. GPIO Troubleshooting Steps

This section introduces the most basic troubleshooting steps for GPIO. If you encounter issues with uncontrollable GPIO input and output, please follow the steps below for troubleshooting.

9.1 Power Confirmation

Measure whether the supply voltage of the GPIO power domain pin is normal and consistent with the design voltage of the hardware schematic? Is the io-domain configuration in the dts correct? The most direct way is to use the io command to confirm the io-domain register. For power domain configuration, please refer to the "IO-Domain Section".

9.2 IO Multiplexing Verification

In most cases, the inability to control the GPIO is due to its multiplexing to another function. A GPIO debug tool can be used to troubleshoot this issue. Generally, the `io` command is used to confirm the GPIO multiplexing registers. For instructions on using the tool, please refer to the "GPIO Debugging Tools" section. Alternatively, a preliminary check can be performed through the `tmp` file in the DTS (Device Tree Source).

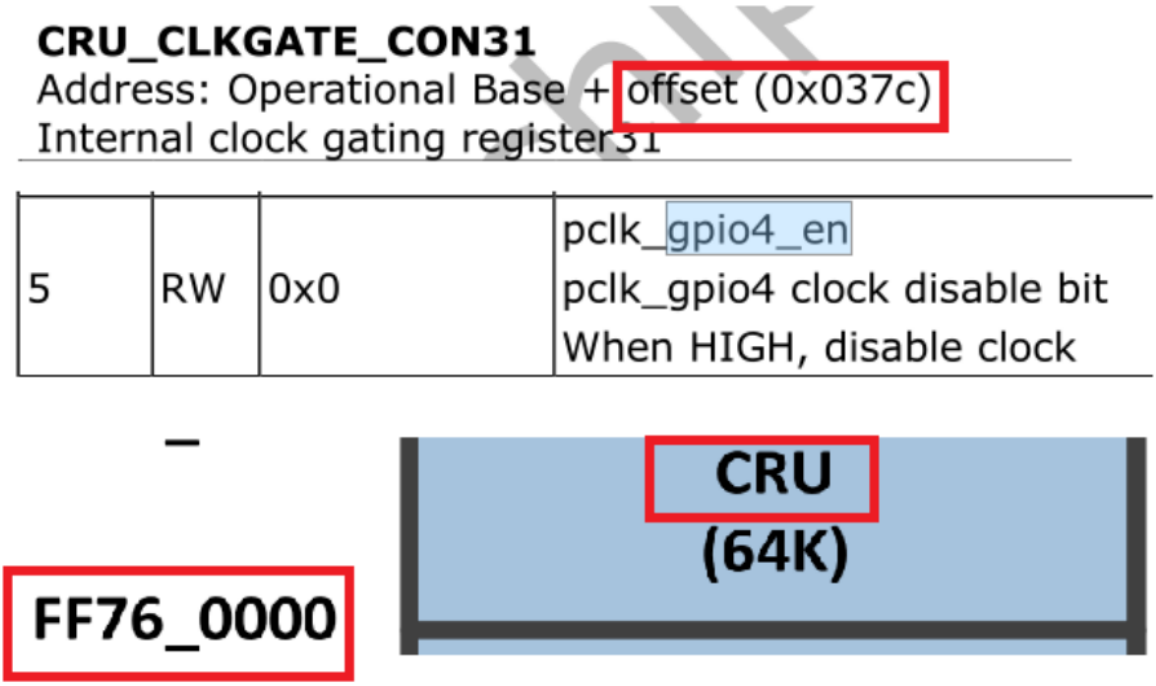
9.3 GPIO CLK Status Confirmation

When the IO tool is used to inspect the GPIO registers, for instance, checking the status of gpio4, the results are as follows:

```
ff790000: 42400000 42400000 42400000 42400000
ff790010: 42400000 42400000 42400000 42400000
ff790020: 42400000 42400000 42400000 42400000
```

```
ff790030: 42400000 42400000 42400000 42400000
ff790040: 42400000 42400000 42400000 42400000
ff790050: 42400000 42400000 42400000 42400000
ff790060: 42400000 42400000 42400000 42400000
ff790070: 42400000 42400000 42400000 42400000
ff790080: 42400000 42400000 42400000 42400000
ff790090: 42400000 42400000 42400000 42400000
ff7900a0: 42400000 42400000 42400000 42400000
ff7900b0: 42400000 42400000 42400000 42400000
ff7900c0: 42400000 42400000 42400000 42400000
ff7900d0: 42400000 42400000 42400000 42400000
ff7900e0: 42400000 42400000 42400000 42400000
ff7900f0: 42400000 42400000 42400000 42400000
```

From the results, it is evident that all values are suspiciously consistent. If the values read out by the io command are partially or entirely displayed as some strange value, this is because the gpio's clk has been turned off, preventing read and write operations. In this case, it is necessary to turn on the clk for gpio4 on the RK3399: (search for gpio4_en in the RK3399 TRM one by one)



First, confirm whether the current gpio4 clk is turned off using the io read command (the register bit is set to high level for disable):

```
io -4 -l  0x100  0xFF76037C
ff76037c: 000001b8 00000010 00000000 0000002a
ff76038c: 00000000 00000000 00000000 00000000
ff76039c: 00000000 00000000 00000000 00000000
ff7603ac: 00000000 00000000 00000000 00000000
ff7603bc: 00000000 00000000 00000000 00000000
ff7603cc: 00000000 00000000 00000000 00000000
ff7603dc: 00000000 00000000 00000000 00000000
ff7603ec: 00000000 00000000 00000000 00000000
ff7603fc: 00000000 00000000 00000000 00000000
```

```
ff76040c: 00000000 00000000 00000000 00000000
ff76041c: 00000000 00000000 00004040 00000000
ff76042c: 00000014 00000000 00000000 00000000
ff76043c: 00000000 00000000 00000000 00000000
ff76044c: 00000000 00000000 00000000 00000000
ff76045c: 00000000 00000000 00000000 00000000
ff76046c: 00000000 00000000 00000000 00000000
```

The result shows `0x000001b8`, which in binary is: `0001 1011 1000`. Therefore, the clk corresponding to gpio4 is the 5th bit, which is the 5th bit from the right when converted to binary, and it is 1 (1 means disable). To enable it, the io command can read and write normally. Use the io write command to turn it on:

```
io -4 -w  0xFF76037C  0x00200000
io -4 -l 0x100 0xFF790000
ff790000: 42000000 6a400000 00000000 00000000
ff790010: 00000000 00000000 00000000 00000000
ff790020: 00000000 00000000 00000000 00000000
ff790030: ffffffff ffffffff 00000000 00000000
ff790040: 00000000 9434d0f9 00000000 00000000
ff790050: 438b2f06 00000000 00000000 00000000
ff790060: 00000000 00000000 00000000 3230372a
ff790070: 00000000 00000000 00000000 00000000
ff790080: 42000000 6a400000 00000000 00000000
ff790090: 00000000 00000000 00000000 00000000
ff7900a0: 00000000 00000000 00000000 00000000
ff7900b0: ffffffff ffffffff 00000000 00000000
ff7900c0: 00000000 9434d0f9 00000000 00000000
ff7900d0: 438b2f06 00000000 00000000 00000000
ff7900e0: 00000000 00000000 00000000 3230372a
ff7900f0: 00000000 00000000 00000000 00000000
```

In addition to writing to the register to turn on the clk, it can also be done through the sys interface, such as:

```
console:sys/kernel/debug/clk # cat clk_summary | grep gpio3
pclk_gpio3         0        1        0   100000000          0     0  50000
console:sys/kernel/debug/clk # ls pclk_gpio3
pclk_gpio3/clk_accuracy         pclk_gpio3/clk_parent
pclk_gpio3/clk_available_parent pclk_gpio3/clk_phase
pclk_gpio3/clk_duty_cycle       pclk_gpio3/clk_prepare_count
pclk_gpio3/clk_enable_count     pclk_gpio3/clk_protect_count
pclk_gpio3/clk_flags            pclk_gpio3/clk_rate
pclk_gpio3/clk_notifier_count

console:sys/kernel/debug/clk # echo 1 > pclk_gpio3/clk_enable_count
console:sys/kernel/debug/clk # cat clk_summary | grep gpio3
    pclk_gpio3          1        2        0   100000000          0     0  50000
```

Note: Whether the gpio clk is turned on depends on whether the corresponding gpio is active (such as during interrupts or data transfer), and is not directly related to whether it is configured or called.

9.4 Conflict Verification Caused by Other Drivers Calling GPIO

For instance, if you want to use an IO pin as a GPIO, you have confirmed the multiplexing is correct as mentioned above, and the power domain is also confirmed to be fine, but the oscilloscope measurement shows that the waveform does not follow your driver code's logic to pull high or low, appearing somewhat "uncontrolled." This could likely be due to other drivers manipulating this GPIO. You can add some debugging information in the `gpio_direction_output` and `gpio_direction_input` interface functions to print out all operations on the GPIO.

```
drivers/gpio/gpiolib.c
int gpiod_direction_input(struct gpio_desc *desc)
{
    + pr_info("%s:  %d\n", __func__, desc_to_gpio(desc));
    ......
}

int gpiod_direction_output(struct gpio_desc *desc, int value)
{
    + pr_info("%s:  %d\n", __func__, desc_to_gpio(desc));
    ......
}
```

Taking RK3588 gpio4b3 as an example to investigate the "uncontrolled" cause. First, confirm the pin base number of gpio4a0 with the following command:

```
root@rk3588-buildroot:/# cat /sys/kernel/debug/gpio
gpiochip0: GPIOs 0-31, parent: platform/fd8a0000.gpio, gpio0:
 gpio-26  (                    |GTP_RST_PORT        ) out hi
 gpio-27  (                    |GTP_INT_IRQ         ) in  hi

gpiochip1: GPIOs 32-63, parent: platform/fec20000.gpio, gpio1:
 gpio-41  (                    |vcc-mipicsi0-regulat) out lo
 gpio-42  (                    |vcc-mipicsi1-regulat) out lo

gpiochip2: GPIOs 64-95, parent: platform/fec30000.gpio, gpio2:
 gpio-76  (                    |reset               ) out hi ACTIVE LOW
 gpio-77  (                    |hdmirx-det          ) in  hi ACTIVE LOW
 gpio-84  (                    |vcc-mipidcphy0-regul) out hi

gpiochip3: GPIOs 96-127, parent: platform/fec40000.gpio, gpio3:
 gpio-96  (                    |bt_default_wake_host) in  lo
 gpio-113 (                    |ircut-open          ) out lo
 gpio-115 (                    |vcc3v3-pcie30       ) out lo

#gpio4a0's pin base number is 128
gpiochip4: GPIOs 128-159, parent: platform/fec50000.gpio, gpio4:
 gpio-130 (                    |reset               ) out hi
 gpio-133 (                    |reset               ) out hi
 gpio-134 (                    |sbu1-dc             ) out lo
 gpio-135 (                    |sbu2-dc             ) out hi
 gpio-136 (                    |vcc5v0-host         ) out hi
```

```
 gpio-137 (                    |enable              ) out lo
 gpio-138 (                    |enable              ) out lo
 gpio-152 (                    |vbus5v0-typec       ) out lo
```

Calculate the pin number for gpio4b3:

```
gpio4b3 = 128 + (B-A) * 8 + 3 = 128 + 1 * 8 + 3 = 139
```

Investigate if there is information about 139:

```
root@rk3588-buildroot:/# dmesg | grep "gpiod_direction_output"
[    1.460212] gpiod_direction_output:  152
[    1.460291] gpiod_direction_output:  115
[    1.460391] gpiod_direction_output:  136
......
[    1.735094] gpiod_direction_output:  133
[    1.806359] gpiod_direction_output:  139  #A driver is controlling gpio4b3
[    2.136271] gpiod_direction_output:  113
```

9.5 GPIO Register Verification

If there are no issues with the IO power domain and IO reset, you can directly read and write the GPIO registers using the `io` command to ultimately confirm whether it is a software configuration issue.

The following three GPIO registers need to be checked:

- Port Data Register: GPIO Output Register (Low/High)
- Port Data Direction Register: GPIO Direction Register (Input/Output)
- **External Port Data Register: GPIO External Port Pin Value (Low/High)**

If a GPIO output is configured to a high level, and after checking, the GPIO output register is configured as 1, the GPIO direction register is configured as 1, but the GPIO external port pin value is 0, it is basically determined to be a hardware issue, where the peripheral circuit has pulled down this GPIO.

10. Frequently Asked Questions

10.1 Inability to Control System Power-On Output Voltage Levels

If there is a requirement for the output to be a high or low voltage level upon power-up, this software cannot control it. It is necessary to select GPIO pins that are pulled up or pulled down by default in hardware. Taking the RK3399 gpio2c2 pin as an example (without external circuits), for GPIOs that have requirements for their default power-on state, the default state can be confirmed through the datasheet or schematic diagram by the suffix letter following the pin name:



If you want to choose a GPIO that defaults to a high voltage level upon power-up, please select GPIO pins with the suffix -u. If you want to choose a GPIO that defaults to a low voltage level upon power-up, please select GPIO pins with the suffix -d.

Note: Some pins are not reused in the GPIO state by default and can be checked through the corresponding TRM document for the default state of the multiplexing register. For example, the JTAG pins, the register is configured by default for JTAG mode.

10.2 Erroneous GPIO Interrupt Triggering

If there is an erroneous triggering of GPIO interrupts, please enable the Schmitt trigger mode by configuring the GPIO as `pcfg_pull_none_smt` mode.

10.3 Inability to Control RV1103/RV1106's gpio3b0~gpio3c3

The gpio3b0~gpio3c3 of RV1103/RV1106 are special; if configured as GPIO mode, they require additional special configuration of related registers and can only be used as inputs (only one mode between GPIO and MIPI for this group of IOs can be active at the same time). Follow these steps:

```
io -4 0xFF3B4804 0x40000000 # Enable gpio3b clock
io -4 0xFF3B4808 0x00100000 # Enable gpio3c clock
io -4 0xFF3E8000 0x7D # Enable Clk/data lane
io -4 0xFF3E8080 0x5F # Enable MIPI/LVDS

# If the pins are gpio3b6~gpio3b7 and gpio3c0~gpio3c, configure the 0xFF3E844C register
io -4 0xFF3E844C 0x00010001 # Switch csi_dphy_path0 to TLL_MODE
# If the pins are gpio3b0~gpio3b5, configure the 0xFF3E884C register
io -4 0xFF3E884C 0x00010001 # Switch csi_dphy_path1 to TLL_MODE

io -4 0xFF55804c 0x00070000 # Set GPIO3_B4 multiplexing register
```

10.4 Inability to Control RV1108's gpio3b0 to gpio3c3

The gpio3b0 to gpio3c3 of RV1108 are special; if configured as GPIO mode, DPHY_LVDS_TTL must be set to TTL mode. The following settings are required:

```
io -4 0x2022838C 0x00000004 #enable TTL mode, disable LVDS mode, disable mipi mode
io -4 0x202283AC 0x000000fd #disable Data Lane, disable Clock Lane
```

10.5 RK3588's gpio4d2 and gpio4d3 Inoperable

Taking RK3588 as an example (other platforms like RK3576 with this issue can also refer to this), if configured as GPIO mode, one should consult the manual force_jtag, as shown in the figure:

SYS_GRF_SOC_CON6

Address: Operational Base + offset (0x0318)

Bit	Attr	Reset Value	Description
31:16	WO	0x0000	write_enable Write enable for lower 16bits, each bit is individual 1'b0: Write access disable 1'b1: Write access enable
15	RW	0x0	pciephy_dtb_sel PCIEPHY DTB select 1'b0: Select PCIE0 PHY DTB output to IO 1'b1: Select PCIE1 PHY DTB output to IO
14	RW	0x1	force_jtag Force SDMMC IO to JTAG function enable When force_jtag is high , gpio4_d2_sel is 0x1 , gpio4_d3_sel is 0x1, and SDMMC_DETECTN is inactive, switch the SDMMC IO to CPU JTAG function.

It is necessary to disable the default force_jtag bit:

```
io -4 0xFD58C318 0x40000000 #disable force_jtag
io -4 0xFD5F8098 0x0F000000 #iomux set to gpio
```

10.6 Android SDK Unable to Locate System-Level Interface for /sys/class/gpio

Modify BOARD_BUILD_GKI := false in the device/rockchip/rk3588/BoardConfig.mk file Enable macros CONFIG_DEBUG_GPIO and CONFIG_GPIO_SYSFS in kernel configuration

10.7 Abnormal GPIO Level Reading

- Is IOMUX configured as GPIO?
- Is the DDR register of GPIO set to input mode?
- Is the GPIO clock enabled?
- Is GPIO internally or externally pulled up?
- Read the GPIO_EXT_PORT register to check the level status (this register reflects the actual pin level, so ensure IOMUX and DDR configurations are correct, and pull-up/pull-down are not in high-impedance state).

10.8 GPIO Interrupt Binding to a Specific Core

Refer to the following patch to modify the interrupt affinity to CPU2:

```
--- a/drivers/gpio/gpio-rockchip.c
+++ b/drivers/gpio/gpio-rockchip.c
@@ -579,6 +579,25 @@ static int rockchip_interrupts_register(struct rockchip_pin_bank *bank)
        irq_set_chained_handler_and_data(bank->irq,
                                         rockchip_irq_demux, bank);

+    // +++ Added: Set interrupt affinity to CPU2 +++
+    {
+        struct cpumask cpumask;
+        int cpu = 2; // Specify CPU2
+
+        // Check if CPU2 is valid
+        if (cpu < num_possible_cpus()) {
+            cpumask_clear(&cpumask);
+            cpumask_set_cpu(cpu, &cpumask);
+            if (irq_set_affinity_hint(bank->irq, &cpumask)) {
+                dev_warn(bank->dev, "Failed to set IRQ affinity for CPU%d\n", cpu);
+            } else {
+                dev_warn(bank->dev, "IRQ%d affinity set to CPU%d\n", bank->irq, cpu);
+            }
+        } else {
+            dev_warn(bank->dev, "Invalid CPU%d for IRQ affinity\n", cpu);
+        }
+    }
+
        return 0;
 }
```

10.9 GPIO Interrupt Response Latency

1. Modify Interrupt Affinity
2. CONFIG_HZ=1000 //Modify heartbeat interval

```
• High HZ value (e.g., 1000):
A clock interrupt is triggered every 1ms, and the scheduler has a maximum delay of 1ms to
respond to new tasks.
• Low HZ value (e.g., 100):
An interrupt is triggered every 10ms, and task switching delay may be as high as 10ms.
```

3. Increase CPU Frequency
4. Apply real-time patches, refer to the documentation inside docs/Patches/Real-Time-Performance to apply the patches.